

Stationary gang scheduling with provable timing guarantees

Seongtae Lee^a, Nan Guan^b, Jinkyu Lee^{c,*}

^a Dept. of Computer Science and Engineering, Sungkyunkwan University, Republic of Korea

^b Dept. of Computer Science, City University of Hong Kong, China

^c Dept. of Computer Science and Engineering, Yonsei University, Republic of Korea

ARTICLE INFO

Keywords:

Gang scheduling

Competitive ratio

Real-time

Safety-critical

Stationary scheduling

ABSTRACT

Safety-critical workloads such as object detection are increasingly deployed on parallel architectures. Real-time gang scheduling manages such workloads by allocating a predefined number of processors to a task simultaneously, so that no thread runs unless all of its threads can run together. We focus on stationary scheduling under fixed-priority (FP), a non-global gang approach that pins each task to a predefined processor set, avoiding the migration overhead and unpredictability of global approaches. While prior work has focused on schedulability analysis, the competitive ratio is also needed to quantify a framework's theoretical resource efficiency. We first show that the state-of-the-art response-time analysis for recursive partitioning scheduling applies to stationary scheduling. Leveraging this, we design a processor-task mapping algorithm from which the first formal competitive ratio for real-time stationary scheduling can be derived: a resource augmentation bound of 5 for a special case and a constant in general. We further show that a simple extension of the mapping preserves this augmentation bound while improving schedulability, and that the resulting framework performs comparably to the only existing stationary approach, which provides no such augmentation bound. To our knowledge, this is the first provably good guarantee for non-global gang scheduling.

1. Introduction

Gang scheduling executes all threads of a parallel task simultaneously across multiple processors. By ensuring that all threads share the same time slot, it minimizes synchronization latency and prevents resource fragmentation, which makes it effective in distributed and parallel processing systems [1–3]. In the real-time domain, threads of parallelizable tasks are bound into gangs on multicore platforms to raise execution rates while guaranteeing timing constraints. This direction has gained renewed attention because safety-critical workloads such as autonomous driving increasingly take parallelizable forms such as Deep Neural Networks (DNNs) and Directed Acyclic Graphs (DAGs) [4]. In these settings, real-time gang scheduling can provide predictable parallel execution and preserve the temporal consistency of sensor data that is otherwise compromised under fragmented thread execution.

Most real-time gang scheduling studies adopt global gang scheduling (GGs), in which a task may run on any set of processors and migrate between processor sets at run time. GGS is flexible, but it is poorly suited to tight timing guarantees. Migration overhead grows with the gang size, and contention on the shared processor pool makes response times difficult to bound. We detail these limitations in Section 2.

To avoid them, we focus on non-global gang scheduling (non-GGS), which statically restricts each task to a fixed processor subset. In particular, we target stationary scheduling (STS) [5], in which each gang

task is permanently mapped to a predefined subset of m_i processors and runs only when it can occupy all of them simultaneously, which offers superior timing predictability and analyzability. Two other non-GGS variants, strictly partitioned scheduling (SPS) [6] and recursive partitioned scheduling (RPS) [7], are reviewed in Section 2. A non-GGS framework combines a processor-task mapping algorithm, which assigns each task to a processor subset, with a schedulability analysis that certifies the resulting assignment, and the mapping is its central design choice. We also observe that the state-of-the-art schedulability analysis developed for RPS applies to STS as well (Section 3), and we use it as the analytical basis of this work.

Most non-GGS studies evaluate a framework only by whether a given task set can be feasibly assigned. Such schedulability-based evaluation does not measure how efficiently the framework uses resources relative to an optimal scheduler. The competitive ratio [8] measures this efficiency. It is a worst-case guarantee, independent of any specific task set, that states how much faster the processors must be for the framework to match an optimal scheduler. Competitive ratios have been studied for uniprocessor [9,10], multiprocessor [11–16], and parallel-task scheduling [17–21], but for gang scheduling (in particular, for non-global gang scheduling), they remain largely unexplored.

* Corresponding author.

E-mail address: jinkyu.lee@yonsei.ac.kr (J. Lee).

<https://doi.org/10.1016/j.sysarc.2026.103983>

Received 8 March 2026; Received in revised form 27 June 2026; Accepted 21 August 2026

Available online 4 September 2026

1383-7621/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

We address this gap by deriving the first formal competitive ratio for fixed-priority STS (STS-FP). The challenge lies not in making the mapping intricate, but in designing one whose worst-case behavior can be bounded against an optimal scheduler. We therefore design a processor-task mapping that assigns each task to a set of sequentially-indexed processors, constructed specifically so that, together with the schedulability analysis above, it admits a provable competitive ratio. The mapping is intentionally simple; its contribution lies in being analyzable rather than elaborate, and this is precisely what makes the bound derivable. For implicit-deadline task sets, it yields a resource augmentation bound of 5 for a special case (denoted $\tau^{\text{RM=PM}}$ and defined later) and a constant bound under a mild condition on the maximum gang size. To the best of our knowledge, this is the first provably good guarantee for non-global gang scheduling.

The main contributions are as follows.

- We show that the state-of-the-art schedulability analysis developed for RPS under fixed-priority, preemptive, exclusive execution carries over to STS-FP, which gives a schedulability test for stationary gang tasks.
- We design a processor-task mapping that assigns each task to a set of sequentially-indexed processors, constructed specifically so that a competitive ratio is derivable; combined with the test above, this establishes the first competitive ratio for non-global gang scheduling: a resource augmentation bound of 5 for $\tau^{\text{RM=PM}}$ task sets and a constant bound for general implicit-deadline task sets under a mild condition on the maximum gang size.
- We present a simple extension of the mapping that lets each task search all processor positions, prove that it retains the competitive ratio, and empirically compare both mappings against the only existing stationary gang scheduling approach, showing that the proposed framework attains comparable schedulability while uniquely providing a provable bound.

The rest of this paper is organized as follows. Section 2 presents the system model, the global and non-global gang scheduling taxonomy, and the competitive ratio. Section 3 establishes the schedulability analysis applicable to STS-FP. Section 4 derives its competitive ratio. Section 5 presents an extended mapping algorithm and an empirical schedulability comparison. Section 6 concludes the paper.

2. Background

In this section we introduce the concepts used throughout the paper. We first define the system model. We then classify gang scheduling into global and non-global approaches and locate STS within this taxonomy. Finally, we introduce the competitive ratio, also called the resource augmentation or speed-up factor, which we use to evaluate resource efficiency.

2.1. System model

We consider a sporadic real-time task system $\tau = \{\tau_1, \tau_2, \dots, \tau_n\}$ of n gang tasks executed on m identical processors, whose set is denoted by $\Pi = \{P_1, P_2, \dots, P_m\}$. Each task $\tau_i = \{T_i, C_i, D_i, m_i\}$ is characterized by a minimum inter-arrival time (period) T_i , a worst-case execution time (WCET) C_i , a relative deadline D_i , and a gang size m_i (the degree of parallelism). All parameters are positive integers. For each τ_i we define its utilization $U_i = C_i/T_i$, and we let $U_{\max} = \max_{\tau_i \in \tau} U_i$ and the total gang utilization $U_{\text{sum}}^{\text{gang}} = \sum_{\tau_i \in \tau} U_i \times m_i$. A task τ_i generates an infinite sequence of jobs whose consecutive release times are separated by at least T_i time units. A job released at t_0 must complete its execution of C_i no later than its absolute deadline $t_0 + D_i$. In the gang scheduling model, each job of τ_i requires simultaneous execution on m_i distinct processors throughout its execution. We assume a fully preemptive model, in which a higher-priority job may preempt a lower-priority one when sufficient processors become available. The tasks are scheduled under a Fixed-Priority (FP) policy. Without loss of generality, smaller indices denote higher priorities, so that $\tau_1 > \tau_2 > \dots > \tau_n$.

2.2. Global and non-global gang scheduling

We classify gang scheduling into global and non-global approaches and examine where STS fits.

Global gang scheduling. Global gang scheduling allows a job of τ_i to execute on any arbitrary set of m_i processors, and most existing research on real-time gang scheduling targets this model [22–30]. Although this approach offers flexibility, it has inherent limitations in both scheduling and schedulability. From the scheduling perspective, when a job that was preempted by a higher-priority job resumes, it is not guaranteed to be assigned the same processors and may resume on a different processor set. The resulting cache invalidation, memory-coherence maintenance, and thread-synchronization overheads typically grow linearly with the gang size m_i . From the schedulability perspective, all gangs compete for the same processor pool, which leads to blocking delays and high temporal unpredictability and makes response-time analysis difficult.

Non-global gang scheduling. Non-global gang scheduling has recently gained attention as a way to mitigate these limitations. First, stationary scheduling (STS) assigns each gang task τ_i to a predefined subset of m_i processors. Let $A_i \subseteq \Pi$ denote the set assigned to τ_i . A job of τ_i executes only when it can simultaneously occupy all processors in A_i , and its execution is confined to A_i . This design reduces scheduling overhead by pinning jobs to the same processors, and it improves schedulability over GGS, because response-time analysis need only account for interference from tasks competing for the same processors. The core of STS is therefore the processor-task mapping algorithm that decides each A_i . A correct mapping must keep all assigned tasks schedulable on their processor sets, so developing an STS framework requires a schedulability analysis that can verify the mapping.

Second, strictly partitioned scheduling (SPS) [6] divides the processor set into disjoint partitions and assigns each gang task to one partition, so that tasks in different partitions share no processors and do not interfere. This removes cross-partition interference and improves schedulability. The main challenge is constructing the partitions. Sun et al. [6] use a first-fit heuristic that considers tasks in descending gang size m_i , placing each task into an existing partition where possible and otherwise creating a new partition of size m_i when the processor budget allows; if it does not, the system is deemed unschedulable. This heuristic is simple and often effective, but it has an inherent limitation, because even a single large-gang task can force a coarse partition and reduce the effectiveness of partitioning.

Last, recursive partitioned scheduling (RPS) [7] generalizes SPS to better handle heterogeneous gang sizes. RPS organizes processors into a hierarchy of partitions, where each partition can be recursively subdivided into disjoint sub-partitions whose processor sets together form that of the parent. Tasks are assigned to levels according to their gang sizes, with large-parallelism tasks placed in higher-level partitions and smaller tasks in finer sub-partitions, which overcomes the SPS limitation above. To exploit this structure, Lee et al. [7] developed a tight schedulability analysis under fixed-priority, preemptive, exclusive-execution scheduling (denoted RPS-FP-EE) that characterizes both direct and indirect interference across partitions. We observe that the RPS-FP-EE analysis does not rely on recursive partitioning itself, but only on the stationary nature of the mapping, so the same analysis applies to STS. In the next section, we make this applicability precise and then present the analysis, which we later use to derive the competitive ratio.

2.3. Competitive ratio

In hard real-time scheduling, the primary question is feasibility, that is, whether every task meets its deadline. Beyond this yes/no test for a specific task set, we often need to quantify how efficiently a scheduling framework uses resources relative to an optimal scheduler across arbitrary workloads. The competitive ratio provides such a guarantee, independent of any specific task set.

Definition 1. Let OPT be an optimal scheduler for a task model on a given platform. The competitive ratio $f \geq 1$ of a scheduling framework A for that model is a constant such that any task set schedulable by OPT on unit-speed processors is also schedulable by A on processors that are f times faster.

Schedulability analysis and the competitive ratio are complementary rather than interchangeable. Schedulability analysis certifies the local feasibility of a specific task set, whereas the competitive ratio is a global, worst-case efficiency guarantee. The two need not agree. For example, GGS can use processor capacity more efficiently than non-GGS but be harder to certify schedulable, so a framework may have a strong competitive ratio despite a pessimistic schedulability test, and vice versa. Relying on either metric alone therefore gives an incomplete picture of a framework. Despite this, the competitive ratio of gang scheduling has remained largely unexplored, and to our knowledge no such result exists for non-global gang scheduling. This paper addresses that gap.

3. A response time analysis for STS-FP

In this section, we present the schedulability analysis originally developed for RPS-FP-EE and prove its applicability to STS-FP, which will then be utilized to derive a competitive ratio.

3.1. Response time analysis of RPS-FP-EE

We first recapitulate recursive partitioned scheduling (RPS) in [7].

Definition 2. Recursive Partitioned Scheduling (RPS) organizes the processor set as a tree of partitions. The root partition is assigned the entire processor set. For any partition ρ in the tree, its child partitions $\{\rho_1, \rho_2, \dots, \rho_t\}$ satisfy the following conditions: the processor sets assigned to the child partitions are pairwise disjoint, and their union equals the processor set of the parent partition, i.e.,

$$A(\rho_i) \cap A(\rho_j) = \emptyset \text{ for all } i \neq j, \quad \bigcup_{i=1}^t A(\rho_i) = A(\rho), \quad (1)$$

where $A(\rho_i)$ denotes the set of processors of ρ_i . Tasks are assigned to one or more partitions within a partition tree and may be executed on the processors associated with the partitions to which they are assigned.

Under RPS-FP-EE, tasks are scheduled with fixed priorities and exclusive execution within the partition hierarchy, such that tasks assigned to partitions that are in an ancestor–descendant relationship are not allowed to be executed simultaneously. As a result, following property holds.

- P1. In RPS-FP-EE, the set of higher-priority tasks that may interfere with τ_k consists of those higher-priority tasks assigned to the same partition or any of its ancestor or descendant partitions.

The schedulability analysis proposed for RPS-FP-EE in [7] leverages this property to categorize higher-priority tasks that can interfere with τ_k into three classes. Let $\Pi(i)$ denote a set of processors of partitions that τ_i is assigned.

- DHP(τ_k) (Directly-interfering Higher-Priority) : the set of higher-priority tasks assigned to either the same partition as τ_k or any of its ancestor or descendant partitions and thus cannot be executed concurrently with τ_k (i.e., they directly interfere with τ_k).

$$\text{DHP}(\tau_k) = \{\tau_i \in \tau \mid i < k \wedge \Pi(i) \cap \Pi(k) \neq \emptyset\} \quad (2)$$

We need to consider the interference of tasks in DHP(τ_k) to compute the response time of τ_k .

- IHP(τ_k) (Indirectly-interfering Higher-Priority) : the set of higher-priority tasks that are not in DHP(τ_k) but belong to the set DHP(τ_i) for at least one task $\tau_i \in \text{DHP}(\tau_k)$ and thus indirectly influence the execution of τ_k . For instance, if $\tau_1 \in \text{DHP}(\tau_2)$, $\tau_2 \in \text{DHP}(\tau_3)$ and $\tau_1 \notin \text{DHP}(\tau_3)$, then τ_1 belongs to IHP(τ_3).

$$\text{IHP}(\tau_k) = \left\{ \tau_i \in \tau \mid i < k \wedge \Pi(i) \cap \Pi(k) = \emptyset \wedge \exists \tau_j \in \text{DHP}(\tau_k) \text{ such that } \Pi(i) \cap \Pi(j) \neq \emptyset \right\} \quad (3)$$

- DHP^{NO-CI}(τ_k) (DHP without carry-in execution) : the set of tasks in DHP(τ_k) that directly interfere with τ_k , but whose critical instant is a synchronous release with τ_k , and therefore do not incur carry-in interference.

$$\text{DHP}^{\text{NO-CI}}(\tau_k) = \bigcup_{\tau_j \in \tau'} \text{DHP}(\tau_j) \cup \{\tau_j\} \setminus \{\tau_k\}, \quad (4)$$

where τ' denotes the set of tasks that satisfies (i) IHP(τ_j) = $\emptyset$ and (ii) DHP(τ_j) $\subseteq$ DHP(τ_k). [7] proves that DHP^{NO-CI}(τ_k) does not incur carry-in interference. We do not repeat the proof in this paper.

The response time of τ_k is calculated with the carry-in/no carry-in interference from higher-priority tasks.

- From [31]: $I_i^{\text{CI}}(\ell)$ denotes the maximum amount of execution of τ_i over any contiguous interval of length ℓ , which occurs when the first job is executed as carry-in to the interval and is computed as follows.

$$I_i^{\text{CI}}(\ell) = \left\lceil \frac{\ell + R_i - C_i}{T_i} \right\rceil \cdot C_i \quad (5)$$

- From [32]: $I_i^{\text{NO-CI}}(\ell)$ denotes the maximum amount of execution of τ_i over any contiguous interval of length ℓ when no job of τ_i is executed as carry-in. In this case, the first job is synchronously released at the beginning of the interval and it is computed as follows.

$$I_i^{\text{NO-CI}}(\ell) = \left\lceil \frac{\ell}{T_i} \right\rceil \cdot C_i \quad (6)$$

Theorem 1. The task set τ is schedulable if, for every task $\tau_k \in \tau$, there exists a response time $R_k \leq D_k$ that satisfies the following equation.

$$R_k = C_k + \sum_{\tau_i \in \text{DHP}(\tau_k)} I_{i \rightarrow k}^*(R_k), \quad (7)$$

where

$$I_{i \rightarrow k}^*(\ell) = \begin{cases} I_i^{\text{NO-CI}}(\ell) = \left\lceil \frac{\ell}{T_i} \right\rceil \cdot C_i, & \text{if } \tau_i \in \text{DHP}^{\text{NO-CI}}(\tau_k), \\ I_i^{\text{CI}}(\ell) = \left\lceil \frac{\ell + R_i - C_i}{T_i} \right\rceil \cdot C_i, & \text{otherwise.} \end{cases} \quad (8)$$

Proof. For task τ_i in DHP^{NO-CI}(τ_k), the synchronous release constitutes a critical instant; thus, the execution time within an interval of length ℓ is upper-bounded by $I_i^{\text{NO-CI}}(\ell)$. Furthermore, since $I_i^{\text{CI}}(\ell)$ is an upper bound on the workload of any task τ_i over an interval of length ℓ , the response time of τ_k is upper-bounded by R_k . $\square$

3.2. RPS-FP-EE as fixed-priority STS

In this subsection, we demonstrate that the schedulability analysis developed for RPS-FP-EE, introduced in Section 3.1, is also applicable to STS-FP (i.e., STS with fixed priority scheduling).

Under STS-FP, tasks are executed only on pre-assigned processors with their fixed-priority. Such that, a task is not allowed to be executed simultaneously with any higher-priority task that shares its assigned processors. This scheduling discipline yields the following property.

- P2. In STS-FP, the set of higher-priority tasks that may interfere with a given task τ_k consists of those higher-priority tasks that share at least one processor with τ_k .

Algorithm 1 Provably good processor-task mapping

```

1:  $y \leftarrow 1$  /* task index */
2: for  $x = 1, 2, 3, \dots, m$  /* processor index */ do
3:   while there exists  $R_y \leq D_y$  that satisfies Eq. (7) for  $\tau_y$  with  $A_y \leftarrow P^{\text{Seq}}(x, m_y)$  do
4:      $A_y \leftarrow P^{\text{Seq}}(x, m_y)$ , store  $R_y$  to calculate  $\{R_i\}$  for  $i > y$ 
5:     if  $y = n$ , then return schedulable
6:      $y \leftarrow y + 1$ 
7:   end while
8: end for
9: Return unschedulable

```

In both RPS-FP-EE and STS-FP scheduling, the higher-priority interfering task is determined by P1 and P2, respectively. Therefore the sets $\text{DHP}(\tau_k)$, $\text{IHP}(\tau_k)$, and $\text{DHP}^{\text{NO-CI}}(\tau_k)$ defined for RPS-FP-EE remain directly applicable to STS-FP by replacing the partition-based processor set $\Pi(i)$ with the task-specific processor set A_i . Consequently, [Theorem 1](#) remains valid and can be applied to any STS-FP.

Lemma 1. Any task set τ that satisfies the condition of [Theorem 1](#) under a given processor-task mapping is schedulable under STS-FP.

Proof. In STS-FP, each task τ_k is statically assigned to a fixed set of processors A_k and cannot be executed concurrently with any higher-priority task τ_i if their assigned processor sets overlap (i.e., $A_i \cap A_k \neq \emptyset$). This is functionally equivalent to the exclusive execution property in RPS-FP-EE, where tasks in an ancestor-descendant relationship are not allowed to be executed simultaneously.

Therefore, by substituting the partition-based processor set $\Pi(i)$ in RPS-FP-EE with the task-specific stationary processor set A_i , the definitions of direct interference (DHP), indirect interference (IHP), and tasks without carry-in ($\text{DHP}^{\text{NO-CI}}$) remain identical in their logical derivation.

Since the response time R_k in [Theorem 1](#) is derived based solely on the workload produced by these interfering task sets, and STS-FP preserves these exact interference relationships, the upper bound on the response time remains valid. Thus, [Theorem 1](#) can be applied to any STS-FP to guarantee schedulability. $\square$

4. Development of competitive ratio of STS

In this section, we propose a processor-task mapping algorithm associated with the proposed schedulability test. We then derive its resource augmentation bound when RM (Rate Monotonic: the smaller period, the higher priority) [33] is applied for implicit-deadline task sets ($T_i = D_i$ holds for every $\tau_i \in \tau$).

4.1. Design of processor-task mapping algorithm

Algorithm 1 explains the processor-task mapping algorithm that assigns a set of sequentially-indexed processors to each task, which necessitates the following definition.

Definition 3. Let $P^{\text{Seq}}(x, m')$ denote a set of m' processors sequentially assigned from the x th processor P_x as follows.

$$P^{\text{Seq}}(x, m') = \begin{cases} \{P_q | x \leq q \leq (x + m' - 1)\}, & \text{if } (x + m' - 1) \leq m, \\ \{P_q | x \leq q \leq m \text{ OR } 1 \leq q \leq (x + m' - 1) \bmod m\}, & \text{otherwise.} \end{cases} \quad (9)$$

$P^{\text{Seq}}(x, m')$ implies that we assign m' ($\leq m$) sequentially-indexed processors starting from P_x . If $(x + m' - 1) \leq m$, we simply assign $P_x, P_{(x+1)}, \dots, P_{(x+m'-1)}$; otherwise, we assign $P_x, P_{(x+1)}, \dots, P_m$ and then assign $P_1, P_2, \dots, P_{(x+m'-1) \bmod m}$.

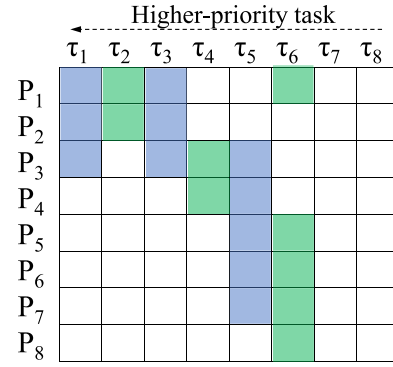


Fig. 1. An example of processor-task mapping for STS.

Now, we explain Algorithm 1 in detail. From the highest-priority task by setting the task index y to 1 (Line 1), the processor index x is sequentially changed from 1 to m (Line 2). Lines 2–8 assign $P^{\text{Seq}}(x, m_y)$ to A_y and store response time of τ_y if assigning $P^{\text{Seq}}(x, m_y)$ to A_y guarantees the schedulability of τ_y according to Eq. (7) (Line 3-4). If the task whose schedulability is guaranteed is the last indexed task, the algorithm returns *schedulable* (Line 5); else, move y to next index of task (Line 6) in order to repeat Lines 3–7 for the next task. On the other hand, assigning $P^{\text{Seq}}(x, m_y)$ to A_y does not guarantee the schedulability of τ_y in Line 3, exit the while loop, meaning that the processor index x is increased by one in Line 2 and repeat Lines 3–7 for the new processor index x with the same task index y . If we iterate to test all m processor indexes without guaranteeing the schedulability of all n tasks, return *unschedulable* by Line 9.

Example 1. Fig. 1 illustrates the processor-mapping by Algorithm 1 when

- τ_1, τ_2 and τ_3 satisfy Eq. (7) for τ_y with $A_y \leftarrow P^{\text{Seq}}(x, m_y)$ for $x = 1$;
- τ_4 does not satisfy Eq. (7) for τ_y with $A_y \leftarrow P^{\text{Seq}}(x, m_y)$ for $x = 1$ and $x = 2$, and satisfies that for $x = 3$;
- τ_5 satisfies Eq. (7) for τ_y with $A_y \leftarrow P^{\text{Seq}}(x, m_y)$ for $x = 3$;
- τ_6 does not satisfy Eq. (7) for τ_y with $A_y \leftarrow P^{\text{Seq}}(x, m_y)$ for $x = 3$ and $x = 4$, and satisfies that for $x = 5$; and
- τ_7 does not satisfy Eq. (7) for τ_y with $A_y \leftarrow P^{\text{Seq}}(x, m_y)$ for $x = 5, 6, 7$ and $x = 8$, and therefore is not shown in the figure.

The following theorem presents the correctness of Algorithm 1 in terms of schedulability.

Theorem 2. If Algorithm 1 returns *schedulable* for a task set τ on m processors, the task set τ whose $\{A_i\}_{\tau_i \in \tau}$ is assigned by Algorithm 1 is schedulable by STS on m processors.

Proof. Suppose that Algorithm 1 returns *schedulable*, but τ whose $\{A_i\}_{\tau_i \in \tau}$ is assigned by Algorithm 1 is not schedulable by STS on m processors. Let $\tau_{y'}$ denote the highest-priority task among the tasks that have a deadline-missed job. Then, $\tau_{y'}$ has the corresponding processor index x' such that Line 3 of Algorithm 1 guarantees the schedulability of $\tau_{y'}$ when $\tau_{y'}$ is assigned to $P^{\text{Seq}}(x', m_{y'})$. Since $R_i \leq D_i$ holds for every $\tau_i \in \text{DHP}(\tau_{y'})$ (by the definition of $\tau_{y'}$), the existence of $\tau_{y'}$ contradicts the correctness of [Theorem 1](#) for $\tau_{y'}$, which proves this theorem. $\square$

Note that the assignment mechanism in Algorithm 1 cannot be said to be novel, in that trial-and-error assignment approaches are intuitive and common and therefore Algorithm 1 is similar to (but not the same as) that of the only existing approach for STS in [5]. However, our assignment algorithms are designed to ease the derivation of a resource augmentation bound to be presented in Section 4.2.

Time-Complexity. To check assigning $P^{\text{Seq}}(x, m_y)$ to A_y guarantees the schedulability of τ_y (Line 3), Eq. (7) needs to know (i) $\text{DHP}(\tau_y)$, $\text{IHP}(\tau_y)$ and $\text{DHP}^{\text{NO-CT}}(\tau_y)$ and (ii) R_j that satisfies Eq. (7). Each time-complexity of (i) and (ii) is $O(n \cdot \max(n, \max_{\tau_k \in \tau} m_k))$ and $O(n \cdot \max_{\tau_k \in \tau} D_k)$ respectively, and the sum of time-complexity for (i) and (ii) is $O(n \cdot \max(n, \max_{\tau_k \in \tau} m_k, \max_{\tau_k \in \tau} D_k))$. This checking (Line 3) happens until y goes from 1 to n and x goes from 1 to m . Therefore, the total time-complexity for run Algorithm 1 is $O(n \cdot (n + m) \cdot \max(n, \max_{\tau_k \in \tau} m_k, \max_{\tau_k \in \tau} D_k))$, which is affordable as the algorithm is performed offline.

Discussion on practical considerations. Algorithm 1 is intentionally kept simple. It assigns each task to a set of sequentially-indexed processors and continues from P_1 after P_m when necessary, so as to expose the structural properties needed for the resource-augmentation analysis in Section 4.2. This simplicity, however, leaves room for practical refinements. First, because tasks are packed onto low-indexed processors whenever possible, the sequential assignment may cluster load on a subset of processors, which can be undesirable for thermal balancing or for leaving contiguous room for subsequently assigned tasks. Second, on platforms with shared caches or non-uniform memory access, an assignment that spans both P_m and P_1 may place a gang across cache, degrading data locality and thereby inflating the worst-case execution times that the analysis takes as input. We emphasize that these effects are orthogonal to the timing analysis itself, which treats the m processors as identical and concerns feasibility rather than micro-architectural overheads: the logical indexing $P_1, \dots, P_m$ can be mapped to the physical topology so that sequentially-indexed sets stay within a cache cluster and only rarely span P_m and P_1 across cluster boundaries. A topology-aware mapping that mitigates clustering and cache fragmentation while preserving the derived competitive ratio is an interesting direction for future work.

4.2. Derivation of resource augmentation bound

Overview of the derivation. Before presenting the details, we outline the overall framework, which follows a resource-augmentation argument in the spirit of [11]. Our goal is to determine how much faster the processors must be for Algorithm 1 to schedule any task set that is feasible on unit-speed processors. We reason by contraposition: assuming that Algorithm 1 returns *unschedulable* for a task set, we show that the task set must impose so large a total demand that it cannot be feasible unless the processors are sufficiently fast. The argument proceeds in four steps. (i) Whenever Algorithm 1 fails, each processor index x ($1 \leq x \leq m$) is associated with a task τ_{y_x} that could not be assigned to a sequentially-indexed processor set starting at P_x ; each such failure yields a necessary overload inequality (Eq. (11)), giving m inequalities in total. (ii) We sum these m inequalities. To bound the sum, we exploit the structural properties of the sequential mapping (Lemma 3) to limit, for each task τ_i , the number of inequalities in which its interference can appear (Lemma 4). (iii) We then upper-bound each per-task interference term by a small multiple of its utilization U_i (Lemma 5), using the fact that the tasks interfering with τ_{y_1} incur no carry-in. (iv) Combining (i)–(iii) shows that an unschedulable task set must satisfy $4 \cdot U_{\text{sum}}^{\text{Gang}} > m \cdot (1 - U_{\text{max}})$ (Lemma 6); its contraposition is a schedulability condition (Theorem 3) that directly yields a resource-augmentation bound of 5 (Theorem 4). We first establish (i)–(iv) for the special task set $\tau^{\text{RM=PM}}$, and then extend the argument to general implicit-deadline task sets by restricting attention to the processors filled before the index first returns to P_1 , which yields a constant bound.

While Algorithm 1 can be applicable to any task-level prioritization policy and both implicit- and constrained-deadline task sets, we derive the resource augmentation bound for the algorithm for implicit-deadline tasks when RM is applied, implying $T_i \leq T_y$ if $i < y$. The overall structure for the derivation is inspired by [11] for RM partitioned scheduling for sequential tasks (equivalent to gang tasks with $m_i = 1$),

but details and difficulty are totally different due to the characteristics of STS.

As a first step, we focus on a special type of task sets in which the task prioritization according to RM is exactly the same as PM (Parallelism Monotonic: the larger m_i , the higher priority). In other words, we consider a task set τ in which $m_i \geq m_j \iff T_i \leq T_j$ holds for any pair of $\tau_i \in \tau$ and $\tau_j \in \tau$; let $\tau^{\text{RM=PM}}$ denote a task set which satisfies such a property. From now on, we present some concepts and properties; some of them hold for a general task set τ , and the others hold for the special task set $\tau^{\text{RM=PM}}$. To this end, we define a term to indicate a task.

Definition 4. Suppose that Algorithm 1 returns *unschedulable* for an implicit-deadline task set τ prioritized by RM. Then, there should exist τ_y that violates the statement in Line 3 under the x th loop of the **for** statement, and therefore exits the **while** loop. We construct a new τ_{y_x} such that its T_{y_x} , C_{y_x} , D_{y_x} and m_{y_x} are set to that of τ_y (i.e., T_y , C_y , D_y and m_y), and assign $P^{\text{Seq}}(x, m_{y_x})$ to A_{y_x} .

For example, τ_{y_x} for every $1 \leq x \leq m = 8$ of the task set in Example 1 is as follows: $\tau_{y_1} = \tau_4$ with $A_{y_1} \leftarrow P^{\text{Seq}}(1, m_4)$, $\tau_{y_2} = \tau_4$ with $A_{y_2} \leftarrow P^{\text{Seq}}(2, m_4)$, $\tau_{y_3} = \tau_6$ with $A_{y_3} \leftarrow P^{\text{Seq}}(3, m_6)$, $\tau_{y_4} = \tau_6$ with $A_{y_4} \leftarrow P^{\text{Seq}}(4, m_6)$, $\tau_{y_5} = \tau_7$ with $A_{y_5} \leftarrow P^{\text{Seq}}(5, m_7)$, $\tau_{y_6} = \tau_7$ with $A_{y_6} \leftarrow P^{\text{Seq}}(6, m_7)$, $\tau_{y_7} = \tau_7$ with $A_{y_7} \leftarrow P^{\text{Seq}}(7, m_7)$, and $\tau_{y_8} = \tau_7$ with $A_{y_8} \leftarrow P^{\text{Seq}}(8, m_7)$.

Since Algorithm 1 returns *unschedulable* only if there exist τ_{y_x} for all $1 \leq x \leq m$, we have m inequalities that violate the schedulability condition in Eq. (7) as follows.

Lemma 2. Suppose that Algorithm 1 returns *unschedulable* for an implicit-deadline task set τ prioritized by RM. Then, the following inequality holds for every $1 \leq x \leq m$,

$$\sum_{\tau_i \in \text{DHP}(\tau_{y_x})} I_{i \rightarrow y_x}^*(T_{y_x}) > T_{y_x} - C_{y_x} \quad (10)$$

$$\begin{aligned} &\iff \sum_{\tau_i \in \text{DHP}(\tau_{y_x})} \frac{I_{i \rightarrow y_x}^*(T_{y_x})}{T_{y_x}} > 1 - \frac{C_{y_x}}{T_{y_x}} \\ &\implies \sum_{\tau_i \in \text{DHP}(\tau_{y_x})} \frac{I_{i \rightarrow y_x}^*(T_{y_x})}{T_{y_x}} > 1 - U_{\text{max}} \end{aligned} \quad (11)$$

Proof. Suppose that Eq. (10) does not hold; then, Eq. (7) holds for a some $R_{y_x} \leq T_{y_x}$, which contradicts the definition of τ_{y_x} . Then, applying $\frac{C_{y_x}}{T_{y_x}} \leq U_{\text{max}}$ to Eq. (10), Eq. (11) holds. $\square$

While the above property holds for a general task set τ , we derive useful properties applicable to the special task set $\tau^{\text{RM=PM}}$.

Lemma 3. Suppose that Algorithm 1 returns *unschedulable* for an implicit-deadline task set $\tau^{\text{RM=PM}}$ prioritized by RM. Then, the following properties hold.

PR1. For τ_{y_1} , $\text{DHP}(\tau_{y_1}) = \{\tau_i | i < y_1\}$ holds, which implies as follows.

$$\sum_{\tau_i \in \text{DHP}(\tau_{y_1})} I_{i \rightarrow y_1}^*(T_{y_1}) = \sum_{\tau_i \in \text{DHP}(\tau_{y_1})} I_i^{\text{NO-CT}}(T_{y_1}) \quad (12)$$

PR2. For τ_{y_x} with $2 \leq x \leq m$ such that $\{P_m, P_1\} \not\subseteq A_{y_x}$, $\text{DHP}(\tau_{y_x}) = \{\tau_i | i < y_x \text{ and } A_i \cap P_x \neq \emptyset\}$ holds.

PR3. For τ_{y_x} with $2 \leq x \leq m$ such that $\{P_m, P_1\} \subseteq A_{y_x}$, $\text{DHP}(\tau_{y_x}) = \{\tau_i | i < y_x \text{ and } A_i \cap P_x \neq \emptyset\} \cup \{\tau_i | i < y_x \text{ and } A_i \cap P_1 \neq \emptyset\}$ holds.

Proof. Write x_i for the starting index of τ_i (so $A_i = P^{\text{Seq}}(x_i, m_i)$). Since the **for**-index of Algorithm 1 is non-decreasing and tasks are assigned in index order, (a) every τ_i with $i < y_x$ has $x_i \leq x$; and for $\tau^{\text{RM=PM}}$ under RM, (b) $i < y_x \Rightarrow m_i \geq m_{y_x}$. Also, (c) no task has $x_i \in \{2, \dots, m_{y_x} - 1\}$: each starting-index-1 task τ_j ($j < y_1$) has $m_j \geq m_{y_x}$

by (b), so $A_j \supseteq \{P_1, \dots, P_{m_{y_x}}\}$; hence retrying the failed τ_{y_1} at any start $s \leq m_{y_x} - 1$ keeps all such τ_j in its window (each contains P_s), so its interference is no smaller than at start 1, where it failed. With (a), no task is placed at such s .

PR1. Each τ_i ($i < y_1$) has $x_i = 1$, so $P_1 \in A_i \cap A_{y_1}$ and $\tau_i \in \text{DHP}(\tau_{y_1})$; thus $\text{DHP}(\tau_{y_1}) = \{\tau_i \mid i < y_1\}$. Each such τ_j has $\text{IHP}(\tau_j) = \emptyset$ and $\text{DHP}(\tau_j) \subseteq \text{DHP}(\tau_{y_1})$, so $\text{DHP}^{\text{NO-Cl}}(\tau_{y_1}) = \text{DHP}(\tau_{y_1})$, giving Eq. (12).

PR2. ($\{P_m, P_1\} \not\subseteq A_{y_x}$) If $\tau_i \in \text{DHP}(\tau_{y_x})$ shares $P_{x'}$ in A_{y_x} , then $x \leq x' \leq m$ and $x_i \leq x$ by (a); as A_i is consecutive from $P_{x'}$, it contains P_x . Hence $\text{DHP}(\tau_{y_x}) = \{\tau_i \mid i < y_x, P_x \in A_i\}$.

PR3. ($\{P_m, P_1\} \subseteq A_{y_x}$, so $A_{y_x} = \{P_x, \dots, P_m\} \cup \{P_1, \dots, P_e\}$, $e \leq m_{y_x} - 1$) Let $\tau_i \in \text{DHP}(\tau_{y_x})$ share $P_{x'}$ in A_{y_x} . By (c), $x_i = 1$ or $x_i \geq m_{y_x}$. If $x_i = 1$, then $P_1 \in A_i$. If $x_i \geq m_{y_x} > e$: for $x' \geq x$, $P_x \in A_i$ as in PR2; for $x' \leq e < x_i$, τ_i reaches $P_{x'}$ only by continuing past P_m , so $P_1 \in A_i$. Hence $\text{DHP}(\tau_{y_x}) = \{\tau_i \mid i < y_x, P_x \in A_i\} \cup \{\tau_i \mid i < y_x, P_1 \in A_i\}$. $\square$

We will sum Eq. (11) for every $1 \leq x \leq m$, which necessitates how many $\frac{I_{i \rightarrow y_x}^*(T_{y_x})}{T_{y_x}}$ terms show up for a given task τ_i . Using the properties in Lemma 3, we can upper-bound the number by one or two depending on whether τ_i belongs to $\text{DHP}(\tau_{y_1})$.

Lemma 4. Suppose that Algorithm 1 returns unschedulable for an implicit-deadline task set $\tau^{\text{RM=PM}}$ prioritized by RM. Then, the following inequality holds for any positive function of τ_i (denoted by $f(\tau_i) > 0$).

$$\begin{aligned} & \sum_{\tau_{y_x} \mid 1 \leq x \leq m} \sum_{\tau_i \in \text{DHP}(\tau_{y_x})} f(\tau_i) \\ & \leq 2 \cdot \sum_{\tau_i \in \text{DHP}(\tau_{y_1})} f(\tau_i) \cdot m_i + \sum_{\tau_i \in \tau \setminus \text{DHP}(\tau_{y_1})} f(\tau_i) \cdot m_i \end{aligned} \quad (13)$$

Proof. By PR1–PR3 in Lemma 3, we will count the number of $f(\tau_i)$ appearing in the LHS of Eq. (13), by considering the following properties.

First, for $\tau_i \in \text{DHP}(\tau_{y_x})$ with $x = 1$, $f(\tau_i)$ may appear once per each τ_{y_x} for PR3. Since $m_i \geq m_j \iff i < j$, the number of such τ_{y_x} is less than any m_i for $\tau_i \in \text{DHP}(\tau_{y_x})$, in total up to m_i times.

Second, in addition to the first case, for $\tau_i \in \text{DHP}(\tau_{y_x})$ for every $1 \leq x \leq m$, $f(\tau_i)$ appears once when A_i includes the corresponding P_x , in total m_i times.

Therefore, $f(\tau_i)$ appears at most $2 \cdot m_i$ times for $\tau_i \in \text{DHP}(\tau_{y_1})$, and at most m_i times for $\tau_i \in \tau \setminus \text{DHP}(\tau_{y_1})$, which proves Eq. (13). $\square$

Then, the following lemma upper-bounds $\frac{I_{i \rightarrow y_x}^*(T_{y_x})}{T_{y_x}}$ in Eq. (11) when $T_i \leq T_y$.

Lemma 5. For an implicit-deadline task set τ prioritized by RM, the following inequalities hold for τ_i and τ_y subject to $i < y$, assuming $R_i \leq T_i$.

$$\frac{I_i^{\text{CI}}(T_y)}{T_y} = \left\lceil \frac{T_y + R_i - C_i}{T_i} \right\rceil \cdot C_i \cdot \frac{1}{T_y} \leq 3 \cdot U_i \quad (14)$$

$$\frac{I_i^{\text{NO-Cl}}(T_y)}{T_y} = \left\lceil \frac{T_y}{T_i} \right\rceil \cdot C_i \cdot \frac{1}{T_y} \leq 2 \cdot U_i \quad (15)$$

$$\frac{I_{i \rightarrow y}^*(T_y)}{T_y} \leq 3 \cdot U_i \quad (16)$$

Proof. Once we apply $T_i \leq T_y \iff \frac{1}{T_i} \geq \frac{1}{T_y} \iff \frac{C_i}{T_i} \geq \frac{C_i}{T_y} \iff U_i \cdot T_y \geq C_i$ and $R_i - C_i \leq T_i$, the followings hold.

$$\begin{aligned} I_i^{\text{CI}}(T_y) &= \left\lceil \frac{T_y + R_i - C_i}{T_i} \right\rceil \cdot C_i \leq \left(1 + \frac{T_y + R_i - C_i}{T_i}\right) \cdot C_i \\ &\leq \left(2 + \frac{T_y}{T_i}\right) \cdot C_i = 2 \cdot C_i + U_i \cdot T_y \leq 3 \cdot U_i \cdot T_y \end{aligned}$$

$$I_i^{\text{NO-Cl}}(T_y) = \left\lceil \frac{T_y}{T_i} \right\rceil \cdot C_i \leq \left(1 + \frac{T_y}{T_i}\right) \cdot C_i = C_i + U_i \cdot T_y \leq 2 \cdot U_i \cdot T_y$$

Therefore, Eqs. (14) and (15) hold, which implies Eq. (16) holds. $\square$

We derive a final inequality by summing Eq. (11) for every $1 \leq x \leq m$ and applying all the lemmas we derived.

Lemma 6. Suppose that Algorithm 1 returns unschedulable for an implicit-deadline task set $\tau^{\text{RM=PM}}$ prioritized by RM. Then, the following inequality should hold.

$$\sum_{\tau_{y_x} \mid 1 \leq x \leq m} \sum_{\tau_i \in \text{DHP}(\tau_{y_x})} \frac{I_{i \rightarrow y_x}^*(T_{y_x})}{T_{y_x}} > m \cdot (1 - U_{\max}) \quad (17)$$

$$\implies 4 \cdot U_{\text{sum}}^{\text{Gang}} > m \cdot (1 - U_{\max}) \quad (18)$$

Proof. By summing Eq. (11) for every $1 \leq x \leq m$, we have Eq. (17). Then, the LHS of Eq. (17) is upper-bounded by Eq. (20), by applying Lemma 4, PR1 in Lemma 3, and Lemma 5 sequentially, which implies that Eq. (18) holds.

$$\sum_{\tau_{y_x} \mid 1 \leq x \leq m} \sum_{\tau_i \in \text{DHP}(\tau_{y_x})} \frac{I_{i \rightarrow y_x}^*(T_{y_x})}{T_{y_x}} \quad (19)$$

$$\begin{aligned} & \leq 2 \cdot \sum_{\tau_i \in \text{DHP}(\tau_{y_1})} \frac{I_{i \rightarrow y_x}^*(T_{y_x})}{T_{y_x}} \cdot m_i + \sum_{\tau_i \in \tau \setminus \text{DHP}(\tau_{y_1})} \frac{I_{i \rightarrow y_x}^*(T_{y_x})}{T_{y_x}} \cdot m_i \\ & = 2 \cdot \sum_{\tau_i \in \text{DHP}(\tau_{y_1})} \frac{I_i^{\text{NO-Cl}}(T_{y_x})}{T_{y_x}} \cdot m_i + \sum_{\tau_i \in \tau \setminus \text{DHP}(\tau_{y_1})} \frac{I_{i \rightarrow y_x}^*(T_{y_x})}{T_{y_x}} \cdot m_i \\ & \leq 2 \cdot \sum_{\tau_i \in \text{DHP}(\tau_{y_1})} 2 \cdot U_i \cdot m_i + \sum_{\tau_i \in \tau \setminus \text{DHP}(\tau_{y_1})} 3 \cdot U_i \cdot m_i \\ & \leq 4 \cdot U_{\text{sum}}^{\text{Gang}} \quad \square \end{aligned} \quad (20)$$

The lemma derives a schedulability condition that depends only on $U_{\text{sum}}^{\text{Gang}}$, $U_{\max}$ and m as follows.

Theorem 3. Consider an implicit-deadline task set $\tau^{\text{RM=PM}}$ such that the tasks are prioritized by RM and $\{A_i\}$ for every $\tau_i \in \tau^{\text{RM=PM}}$ is assigned by Algorithm 1. Then, $\tau^{\text{RM=PM}}$ is schedulable by STS on m processors, if the following inequality holds.

$$\frac{4 \cdot U_{\text{sum}}^{\text{Gang}}}{1 - U_{\max}} \leq m \quad (21)$$

Proof. We prove the contraposition of the theorem. Suppose that Algorithm 1 returns unschedulable for $\tau^{\text{RM=PM}}$. By Lemma 6, Eq. (18) holds, which is the negation of Eq. (21). Therefore, the contraposition is if Eq. (21) holds, Algorithm 1 returns schedulable, which proves the Theorem by Theorem 2. $\square$

Then, the resource augmentation bound for Algorithm 1 for $\tau^{\text{RM=PM}}$ is derived as follows.

Theorem 4. If it is feasible to schedule an implicit-deadline task set $\tau^{\text{RM=PM}}$ on m unit-capacity processors without any job deadline miss, $\tau^{\text{RM=PM}}$ is schedulable by STS associated with RM (for the task prioritization) and Algorithm 1 (for the processor-task mapping) on m processors each of whose capacity is 5 units.

Proof. Suppose that τ is feasible on m processors each of whose speed is ξ . Then, $U_{\text{sum}}^{\text{Gang}} \leq m \cdot \xi$ and $U_{\max} \leq \xi$ should hold [11]. Therefore, the following inequality proves the theorem.

$$\begin{aligned} \frac{4 \cdot U_{\text{sum}}^{\text{Gang}}}{1 - U_{\max}} \leq m & \iff \frac{4 \cdot m \cdot \xi}{1 - \xi} \leq m \\ & \iff 4 \cdot m \cdot \xi \leq m - m \cdot \xi \\ & \iff 5 \cdot m \cdot \xi \leq m \\ & \iff \frac{1}{\xi} \geq 5 \quad \square \end{aligned} \quad (22)$$

Based on the results for a special task set $\tau^{\text{RM=PM}}$ so far, we now derive the resource augmentation bound for Algorithm 1 for a general task set τ . To this end, we define a term.

Definition 5. Suppose that Algorithm 1 returns *unschedulable* for an implicit-deadline task set τ prioritized by RM. Let y^* denote the task index of the first τ_y (but not τ_1) such that $P^{\text{Seq}}(x, m_y)$ in Line 3 includes both P_m and P_1 ; let x^* denote the processor index x for the situation.

Then, instead of focusing on τ_{y_x} for every $1 \leq x \leq m$ for $\tau^{\text{RM=PM}}$, we focus on τ_{y_x} for every $1 \leq x \leq x^* - 1$ for τ .

Lemma 7. Suppose that Algorithm 1 returns *unschedulable* for an implicit-deadline task set τ prioritized by RM. Then, the following properties hold.

PR4. For τ_{y_x} with $1 \leq x \leq x^* - 1$, $\text{DHP}(\tau_{y_x}) = \{\tau_i | i < y_x \text{ and } A_i \cap P_x \neq \emptyset\}$ holds.

Proof. By the definition of x^* , τ_{y_x} with $1 \leq x \leq x^* - 1$ satisfies either $\{P_m, P_1\} \not\subseteq A_{y_x}$ or $x = 1$. Therefore, the proof is the same as that of PR3 and PR2 in Lemma 3, respectively for the former and the latter. $\square$

We will sum Eq. (11) for every $1 \leq x \leq x^* - 1$, which necessitates how many $\frac{I_{i \rightarrow y_x}^*(T_{y_x})}{T_{y_x}}$ terms show up for a given task τ_i . Using PR4 in Lemma 7, we can upper-bound the number by one.

Lemma 8. Suppose that Algorithm 1 returns *unschedulable* for an implicit-deadline task set τ prioritized by RM. Then, the following inequality holds for any positive function of τ_i (denoted by $f(\tau_i) > 0$).

$$\sum_{\tau_{y_x} | 1 \leq x \leq x^* - 1} \sum_{\tau_i \in \text{DHP}(\tau_{y_x})} f(\tau_i) \leq \sum_{\tau_i \in \tau} f(\tau_i) \cdot m_i \quad (23)$$

Proof. If we apply PR4 in Lemma 7, the proof is similar to that of Lemma 4. $\square$

We derive a final inequality by summing Eq. (11) for every $1 \leq x \leq x^* - 1$ and applying all the lemmas we derived.

Lemma 9. Suppose that Algorithm 1 returns *unschedulable* for an implicit-deadline task set τ prioritized by RM. Then, the following inequality should hold.

$$\sum_{\tau_{y_x} | 1 \leq x \leq x^* - 1} \sum_{\tau_i \in \text{DHP}(\tau_{y_x})} \frac{I_{i \rightarrow y_x}^*(T_{y_x})}{T_{y_x}} > (x^* - 1) \cdot (1 - U_{\max}) \quad (24)$$

$$\implies 3 \cdot U_{\text{sum}}^{\text{Gang}} > (x^* - 1) \cdot (1 - U_{\max}) \quad (25)$$

$$\implies 3 \cdot U_{\text{sum}}^{\text{Gang}} > (m - \max_{\tau_i \in \tau} m_i + 1) \cdot (1 - U_{\max}) \quad (26)$$

Proof. By summing Eq. (11) for every $1 \leq x \leq x^* - 1$, we have Eq. (24). Similar to the proof of Lemma 6, if we apply Lemma 8 and Lemma 5 to Eq. (24), we have Eq. (25).

Suppose that $x^* + \max_{\tau_i \in \tau} m_i - 1 \geq m + 1$ does not hold. Then, it immediately contradicts the definition of y^* . Therefore, $x^* + \max_{\tau_i \in \tau} m_i - 1 \geq m + 1 \iff x^* - 1 \geq m - \max_{\tau_i \in \tau} m_i + 1$, thereby Eq. (26) from Eq. (25). $\square$

Theorem 5. Consider an implicit-deadline task set τ such that the tasks are prioritized by RM and $\{A_i\}$ for every $\tau_i \in \tau$ is assigned by Algorithm 1. Then, τ is schedulable by STS on m processors, if the following inequality holds.

$$\frac{3 \cdot U_{\text{sum}}^{\text{Gang}}}{1 - U_{\max}} \leq m - \max_{\tau_i \in \tau} m_i + 1 \quad (27)$$

Proof. The proof is the same as that of Theorem 3. $\square$

Then, the resource augmentation bound for Algorithm 1 for τ is derived as follows.

Theorem 6. If it is feasible to schedule an implicit-deadline task set τ on m unit-capacity processors without any job deadline miss, τ is schedulable by STS associated with RM (for the task prioritization) and Algorithm 1 (for the processor-task mapping) on m processors each of whose capacity is $\frac{1}{\xi} \geq \frac{4 \cdot m - \max_{\tau_i \in \tau} m_i + 1}{m - \max_{\tau_i \in \tau} m_i + 1}$ units.

Proof. Suppose that τ is feasible on m processors each of whose speed is ξ . Then, $U_{\text{sum}}^{\text{Gang}} \leq m \cdot \xi$ and $U_{\max} \leq \xi$ should hold [11]. Therefore, the following inequality proves the theorem.

$$\begin{aligned} & \frac{3 \cdot U_{\text{sum}}^{\text{Gang}}}{1 - U_{\max}} \leq m - \max_{\tau_i \in \tau} m_i + 1 \\ \iff & \frac{3 \cdot m \cdot \xi}{1 - \xi} \leq m - \max_{\tau_i \in \tau} m_i + 1 \\ \iff & 3 \cdot m \cdot \xi \leq (m - \max_{\tau_i \in \tau} m_i + 1) - (m - \max_{\tau_i \in \tau} m_i + 1) \cdot \xi \\ \iff & (4 \cdot m - \max_{\tau_i \in \tau} m_i + 1) \cdot \xi \leq (m - \max_{\tau_i \in \tau} m_i + 1) \\ \iff & \frac{1}{\xi} \geq \frac{4 \cdot m - \max_{\tau_i \in \tau} m_i + 1}{m - \max_{\tau_i \in \tau} m_i + 1} = 4 + \frac{3 \cdot \max_{\tau_i \in \tau} m_i - 3}{m - \max_{\tau_i \in \tau} m_i + 1} \quad \square \quad (28) \end{aligned}$$

Note that the above resource augmentation bound is upper-bounded by a constant if $\max_{\tau_i \in \tau} m_i$ is a proper function of m , e.g., $a \cdot m$ where $a < 1.0$. For example, if $\max_{\tau_i \in \tau} m_i = \frac{m}{2}$, then $\frac{1}{\xi} \geq \frac{4 \cdot m - \frac{m}{2} + 1}{m - \frac{m}{2} + 1} = \frac{\frac{7}{2} \cdot (m+2)}{\frac{1}{2} \cdot (m+2)} = 7 - \frac{12}{m+2}$ holds, implying the resource augmentation bound is upper-bounded by 7. Tightening this constant bound is left as future work.

5. Extended processor-task mapping and schedulability comparison

The processor-task mapping of Algorithm 1 is deliberately restrictive. It assigns every task a set of sequentially-indexed processors that starts from a single index x shared by all tasks, and this index only increases. As explained in Section 4.2, that restriction is what exposes the structural property used to derive the resource augmentation bound. The same restriction, however, makes Algorithm 1 give up on a task as soon as the shared index reaches m , even when the task would fit at an earlier starting position that a higher-priority task has already passed.

This section has two parts. We first present a simple extension of Algorithm 1, denoted by Algorithm 2, that lets each task search all m starting positions on its own. The extension dominates Algorithm 1 in terms of schedulability and therefore inherits its resource augmentation bound, while reclaiming the schedulability that the shared index gives up. We then compare Algorithm 1, Algorithm 2, and the only existing stationary scheduling approach on randomly-generated task sets.

5.1. Extended processor-task mapping algorithm

Although Algorithm 1 yields the first provably good mapping for stationary gang scheduling, it does not use its full schedulability potential. When the shared index x reaches m and the current task τ_y still fails Eq. (7), Algorithm 1 returns *unschedulable* in its last line, even though τ_y might satisfy Eq. (7) at an earlier start $P^{\text{Seq}}(x', m_y)$ with $x' < x$. Example 1 illustrates this situation, where τ_7 fails at every start from $x = 5$ to $x = 8$ that the shared index reaches and is therefore rejected, while the earlier starts $x = 1$ to $x = 4$ that higher-priority tasks have passed are never reconsidered.

Motivated by this, Algorithm 2 extends Algorithm 1 so that every task τ_y is allowed to try $A_y = P^{\text{Seq}}(x, m_y)$ for all m starting positions. The single for loop over x in Algorithm 1, which is shared across tasks, becomes a per-task search. After a task is placed, the search continues from the same index for the next task (Line 5), and whenever a task τ_y fails at the current index, the index advances cyclically (Line 12) while the counter Fail_y records how many starts τ_y has tried (Line 10).

Algorithm 2 Extended processor-task mapping

```

1:  $y \leftarrow 1$  /* task index */
2:  $\text{Fail}_i \leftarrow 0$  for  $1 \leq i \leq n$ 
3:  $x \leftarrow 1$  /* processor index */
4: while true do
5:   while there exists  $R_y \leq D_y$  that satisfies Eq. (7) for  $\tau_y$  with  $A_y \leftarrow P^{\text{Seq}}(x, m_y)$  do
6:      $A_y \leftarrow P^{\text{Seq}}(x, m_y)$ , store  $R_y$  to calculate  $\{R_i\}$  for  $i > y$ 
7:     if  $y = n$ , then return schedulable
8:      $y \leftarrow y + 1$ 
9:   end while
10:   $\text{Fail}_y \leftarrow \text{Fail}_y + 1$ 
11:  if  $\text{Fail}_y = m$  then return unschedulable
12:   $x \leftarrow (x \bmod m) + 1$ 
13: end while

```

Algorithm 2 returns *unschedulable* only when some task τ_y has failed at all m starting positions, that is, $\text{Fail}_y = m$ in Line 11.

The correctness of Algorithm 2 follows in the same way as that of Algorithm 1.

Theorem 7. *If Algorithm 2 returns schedulable for a task set τ on m processors, the task set τ whose $\{A_i\}_{\tau_i \in \tau}$ is assigned by Algorithm 2 is schedulable by STS on m processors.*

Proof. The argument is identical to that of Theorem 2. Each task τ_y is placed only at a start $P^{\text{Seq}}(x, m_y)$ for which Eq. (7) guarantees $R_y \leq D_y$, and the response times of higher-priority tasks are fixed at the time of placement. Hence $R_i \leq D_i$ holds for every $\tau_i \in \text{DHP}(\tau_y)$, and the schedulability of the resulting assignment follows from the correctness of Theorem 1. $\square$

The following lemma states the relation between the two algorithms and its consequence for the resource augmentation bound.

Lemma 10. *For a task set τ on m processors, if Algorithm 1 returns schedulable, then Algorithm 2 also returns schedulable. Consequently, the resource augmentation bound established for Algorithm 1 in Section 4.2 is also a resource augmentation bound for Algorithm 2.*

Proof. Until Algorithm 1 returns *unschedulable*, the two algorithms make the same placements. Both start from $x = 1$, both assign each task to the first start at which Eq. (7) is satisfied, and both advance the starting index only when the current task fails. The index of Algorithm 2 returns from m to 1 only after the current task has failed at $x = m$, which is exactly the point at which Algorithm 1 returns *unschedulable*. Therefore every assignment produced by Algorithm 1 is also produced by Algorithm 2, which gives the dominance in terms of schedulability. The bound carries over by this dominance. If a speed-up of ξ makes every feasible task set schedulable by Algorithm 1, the same speed-up makes it schedulable by Algorithm 2, so the bound of Theorem 4 and the conditions of Theorems 3 and 5 hold for Algorithm 2 as well. $\square$

Time-Complexity. Algorithm 2 uses the same feasibility check as Algorithm 1 in Line 5, whose time-complexity is $O(n \cdot \max(n, \max_{\tau_k \in \tau} m_k, \max_{\tau_k \in \tau} D_k))$. This check is performed at most m times for each of the n tasks, so the total time-complexity of Algorithm 2 is $O(n^2 \cdot m \cdot \max(n, \max_{\tau_k \in \tau} m_k, \max_{\tau_k \in \tau} D_k))$, which remains affordable because the mapping is computed offline.

5.2. Schedulability comparison

We compare the schedulability of Algorithm 1, Algorithm 2, and the existing stationary gang scheduling approach on randomly-generated task sets.

Table 1

Ratio of task sets deemed schedulable by each approach, normalized by UGB (%). Columns are implicit (I) or constrained (C) deadlines, each with low (L, $m_i \in [1, m/2]$) or high (H, $m_i \in [1, m]$) parallelism.

Approach	Implicit (I)		Constrained (C)	
	L	H	L	H
<i>m</i> = 8 processors				
Algorithm 1	89.4	96.5	87.9	96.0
Algorithm 2	93.2	97.7	92.4	97.2
UGB	100.0	100.0	100.0	100.0
<i>m</i> = 16 processors				
Algorithm 1	86.9	96.5	84.6	95.5
Algorithm 2	88.5	97.1	86.6	96.3
UGB	100.0	100.0	100.0	100.0

Task Set Generation. We generate gang task sets by the UUniFast method [34], following the procedure used in recent stationary gang scheduling studies [5,7]. We consider $m \in \{8, 16\}$ processors and, for each m , task counts $n \in \{m, \frac{3}{2}m, 2m, \frac{5}{2}m\}$. Each task set is characterized by two binary parameters. Each m_i is drawn uniformly from $[1, m/2]$ (denoted L) or from $[1, m]$ (denoted H), and deadlines are either implicit, $D_i = T_i$ (denoted I), or constrained, $D_i \leq T_i$ (denoted C). The normalized gang utilization $U_{\text{sum}}^{\text{gang}}/m$, where $U_{\text{sum}}^{\text{gang}} = \sum_{\tau_i \in \tau} (C_i/T_i) \cdot m_i$, is varied from 0.1 to 1.0 in steps of 0.1. At each point, the per-task gang utilizations $(C_i/T_i) \cdot m_i$ are drawn by UUniFast so that they sum to $U_{\text{sum}}^{\text{gang}}$, subject to $C_i/T_i \leq 1$. Periods T_i are drawn uniformly from $[100, 1000]$, the worst-case execution time C_i is set accordingly, and for constrained deadlines D_i is drawn uniformly from $[C_i, T_i]$. All approaches use the deadline-monotonic priority order, which coincides with rate-monotonic for implicit-deadline task sets. For each of the $2 \times 4 \times 2 \times 2 = 32$ configurations and each of the 10 utilization points, we generate 1,000 task sets and report the fraction deemed schedulable, yielding 320,000 task sets in total on which all approaches are compared.

Compared Approaches. We compare the following three approaches for stationary gang scheduling. The first two use the schedulability analysis of Section 4.

- Algorithm 1: the provably good mapping with the shared, monotone starting index.
- Algorithm 2: its extension that lets each task search all m starting positions.
- UGB: the only existing stationary gang scheduling approach [5], which maps each task independently to the first feasible of its m sliding-window placements and tests schedulability with a self-suspension-based analysis.

We restrict the comparison to stationary scheduling, which is the model studied in this paper. Global gang scheduling, in which a task may occupy any m_i processors at run time, is a different model with an unrestricted assignment space, and we do not include it here.

Results. Fig. 2 reports the schedulable ratio of the three approaches as a function of $U_{\text{sum}}^{\text{gang}}/m$. At low utilization, all three approaches schedule every task set; at high utilization, none does. Therefore, the approaches differ only over a middle band of utilizations. Algorithm 1 is the most conservative there, which is the intended cost of its shared, monotone starting index. Algorithm 2 recovers a large part of this gap and stays close to UGB, and the three curves nearly coincide for the H configurations, where wide tasks leave few distinct starting positions for the extension to exploit. Table 1 summarizes the same data as the fraction of task sets each approach deems schedulable, normalized by UGB. Across the configurations, Algorithm 1 covers between 84.6% and 96.5% of what UGB covers, and Algorithm 2 raises this to between 86.6% and 97.7%, with the largest improvement in the low-parallelism (L) configurations.

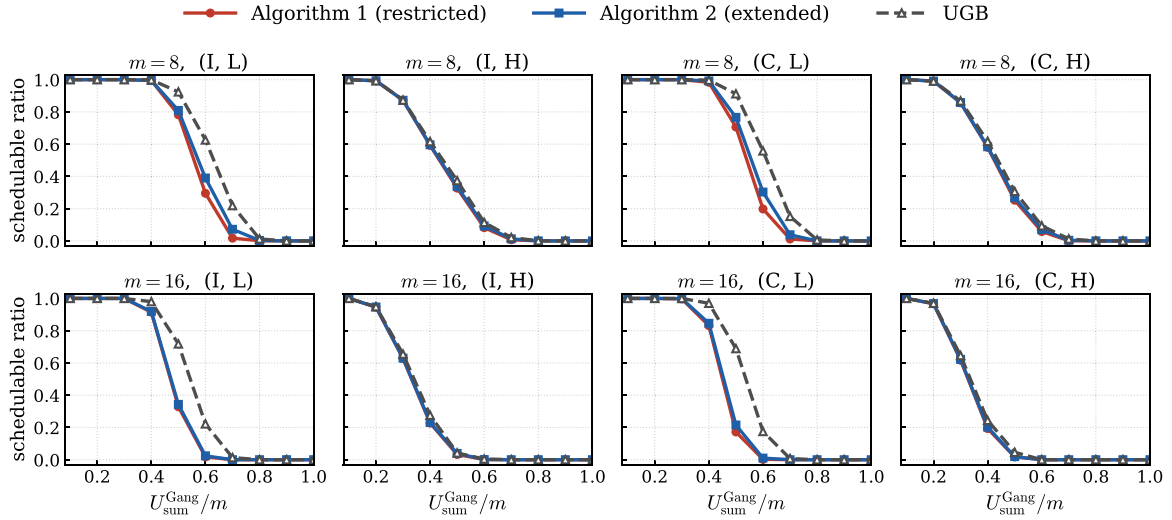


Fig. 2. Ratio of task sets deemed schedulable by Algorithm 1, Algorithm 2, and the existing stationary approach UGB [5], as a function of the normalized gang utilization U_{sum}^{Gang}/m . I and C denote implicit and constrained deadlines, and L and H denote $m_i \in [1, m/2]$ and $m_i \in [1, m]$, respectively.

Schedulability beyond UGB. Algorithm 2 and UGB are not in a dominance relation: there exist task sets that Algorithm 2 deems schedulable while UGB does not. Among the task sets schedulable by at least one of the two approaches, 0.7% are schedulable by Algorithm 2 but not by UGB, and this fraction rises to about 1.0% in the low-parallelism (L) configurations. Beyond providing a resource augmentation bound, the proposed framework therefore also schedules task sets that the existing stationary approach cannot.

Discussion. UGB schedules a moderately larger fraction of task sets than the proposed mapping, and the comparison should be read in that light. As noted in Section 2, the competitive ratio and empirical schedulability are complementary measures: the former is a worst-case guarantee that holds for every task set, whereas the latter is specific to a generated workload, and a strong value of one does not imply the other. The distinguishing property of the proposed framework is that it admits a competitive ratio at all. To the best of our knowledge, it is the only stationary gang scheduling approach with a provable resource-augmentation bound, whereas UGB provides none. The proposed framework therefore does not aim to surpass UGB in empirical schedulability; it provides a worst-case efficiency guarantee that UGB cannot, at a modest schedulability cost. Concretely, Algorithm 1 fixes a single starting index that is shared by all tasks and never decreases, which is exactly the structural property used in Section 4.2 to derive the bound, but which also forgoes placements that UGB is free to use. UGB searches every task's m placements independently and keeps no such structure, which is why it admits no resource-augmentation bound. The extended Algorithm 2 lifts the restriction, so it matches UGB closely while, by Lemma 10, still inheriting the bound of Algorithm 1.

We finally remark that more elaborate stationary frameworks, such as the recursive partitioning of [7], can attain higher schedulability than the approaches considered here. Such frameworks are designed for schedulability performance and do not derive a resource augmentation bound, so they pursue a different goal. Narrowing the remaining gap while retaining a constant resource augmentation bound is an interesting direction for future work.

6. Conclusion

This paper investigates the theoretical resource efficiency of stationary scheduling (STS), a non-global gang approach. To address this, we first established that existing state-of-the-art schedulability analysis for

recursive partitioning scheduling is directly applicable to STS. Based on this foundation, we proposed an intuitive processor-task mapping algorithm and derived the first formal competitive ratio and resource augmentation bounds for STS under fixed-priority scheduling. Our analysis proves a resource augmentation bound of 5 for special cases (RM=PM) and a constant bound under general conditions. We further showed that a simple extension of the mapping retains this bound, and an empirical comparison confirmed that the resulting framework attains schedulability comparable to the only existing stationary gang scheduling approach while uniquely providing a provable bound. These results demonstrate that STS is theoretically robust.

CRedit authorship contribution statement

Seongtae Lee: Writing – original draft. **Nan Guan:** Writing – review & editing. **Jinkyu Lee:** Methodology, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (RS-2024-00438248, RS-2024-00460389, RS-2025-17622972, RS-2025-22502968). This research was also supported by the Yonsei University Research Fund of 2025 (2025-22-0408).

Data availability

Data will be made available on request.

References

- [1] D.G. Feitelson, L. Rudolph, Distributed hierarchical control for parallel processing, *Computer* 23 (5) (1990) 65–77.
- [2] D.G. Feitelson, M.A. Jette, Improved utilization and responsiveness with gang scheduling, in: *Workshop on Job Scheduling Strategies for Parallel Processing*, Springer, 1997, pp. 238–261.
- [3] H.D. Karatza, Scheduling gangs in a distributed system, *Int. J. Simul.: Syst. Sci. Technol. UK Simul. Soc.* 7 (1) (2006) 15–22.
- [4] W. Kwon, G.-I. Yu, E. Jeong, B.-G. Chun, Nimble: Lightweight and parallel gpu task scheduling for deep learning, *Adv. Neural Inf. Process. Syst.* 33 (2020) 8343–8354.
- [5] N. Ueter, M. Günzel, G. von der Brüggen, J.-J. Chen, Hard real-time stationary GANG-scheduling, in: *Proceedings of Euromicro Conference on Real-Time Systems*, 2021, pp. 10:1–10:19.
- [6] B. Sun, T. Kloda, M. Caccamo, Strict partitioning for sporadic rigid gang tasks, in: *2024 IEEE 30th Real-Time and Embedded Technology and Applications Symposium, RTAS, IEEE*, 2024, pp. 252–264.
- [7] S. Lee, N. Guan, J. Lee, Recursive partitioned scheduling for real-time gang tasks, in: *2025 IEEE Real-Time Systems Symposium, RTSS, IEEE*, 2025, pp. 135–147.
- [8] B. Kalyanasundaram, K. Pruhs, Speed is as powerful as clairvoyance, *J. ACM* 47 (4) (2000) 617–643.
- [9] R. Davis, S. Baruah, T. Rothvoß, A. Burns, Quantifying the sub-optimality of uniprocessor fixed priority pre-emptive scheduling for sporadic tasksets with arbitrary deadlines, in: *17th International Conference on Real-Time and Network Systems*, 2009, pp. 23–34.
- [10] R.I. Davis, T. Rothvoß, S.K. Baruah, A. Burns, Exact quantification of the sub-optimality of uniprocessor fixed priority pre-emptive scheduling, *Real-Time Syst.* 43 (3) (2009) 211–258.
- [11] N. Fisher, S. Baruah, T.P. Baker, The partitioned scheduling of sporadic tasks according to static-priorities, in: *ECRTS*, 2006, pp. 118–127.
- [12] J.-J. Chen, S. Chakraborty, Resource augmentation bounds for approximate demand bound functions, in: *2011 IEEE 32nd Real-Time Systems Symposium, IEEE*, 2011, pp. 272–281.
- [13] R.L. Graham, Bounds on multiprocessing timing anomalies, *SIAM J. Appl. Math.* 17 (2) (1969) 416–429.
- [14] D.S. Hochbaum, D.B. Shmoys, Using dual approximation algorithms for scheduling problems theoretical and practical results, *J. ACM* 34 (1) (1987) 144–162.
- [15] A. Karrenbauer, T. Rothvoß, A 3/2-approximation algorithm for rate-monotonic multiprocessor scheduling of implicit-deadline tasks, in: *International Workshop on Approximation and Online Algorithms*, Springer, 2010, pp. 166–177.
- [16] J.-J. Chen, Partitioned multiprocessor fixed-priority scheduling of sporadic real-time tasks, in: *2016 28th Euromicro Conference on Real-Time Systems, ECRTS, IEEE*, 2016, pp. 251–261.
- [17] A. Saifullah, J. Li, K. Agrawal, C. Lu, C. Gill, Multi-core real-time scheduling for generalized parallel task models, *Real-Time Syst.* 49 (4) (2013) 404–435.
- [18] B. Andersson, D. de Niz, Analyzing global-EDF for multiprocessor scheduling of parallel tasks, in: *International Conference on Principles of Distributed Systems*, Springer, 2012, pp. 16–30.
- [19] S. Baruah, V. Bonifaci, A. Marchetti-Spaccamela, L. Stougie, A. Wiese, A generalized parallel task model for recurrent real-time processes, in: *2012 IEEE 33rd Real-Time Systems Symposium, IEEE*, 2012, pp. 63–72.
- [20] V. Bonifaci, A. Marchetti-Spaccamela, S. Stiller, A. Wiese, Feasibility analysis in the sporadic DAG task model, in: *2013 25th Euromicro Conference on Real-Time Systems, IEEE*, 2013, pp. 225–233.
- [21] X. Jiang, N. Guan, D. Liu, W. Liu, Analyzing GEDF scheduling for parallel real-time tasks with arbitrary deadlines, in: *2019 Design, Automation & Test in Europe Conference & Exhibition, DATE, IEEE*, 2019, pp. 1537–1542.
- [22] S. Kato, Y. Ishikawa, Gang EDF scheduling of parallel task systems, in: *2009 30th IEEE Real-Time Systems Symposium, IEEE*, 2009, pp. 459–468.
- [23] Z. Dong, K. Yang, N. Fisher, C. Liu, Tardiness bounds for sporadic gang tasks under preemptive global EDF scheduling, *IEEE Trans. Parallel Distrib. Syst.* 32 (12) (2021) 2867–2879.
- [24] Z. Dong, C. Liu, A utilization-based test for non-preemptive gang tasks on multiprocessors, in: *2022 IEEE Real-Time Systems Symposium, RTSS, IEEE*, 2022, pp. 105–117.
- [25] S. Lee, S. Lee, J. Lee, Response time analysis for real-time global gang scheduling, in: *2022 IEEE Real-Time Systems Symposium, RTSS, IEEE*, 2022, pp. 92–104.
- [26] S. Lee, N. Guan, J. Lee, Design and timing guarantee for non-preemptive gang scheduling, in: *2022 IEEE Real-Time Systems Symposium, RTSS, IEEE*, 2022, pp. 132–144.
- [27] F.A. Silva, E.P. Lopes, E.P. Aude, F. Mendes, J. Silveira, H. Serdeira, M. Martins, W. Cirne, Response time analysis of gang scheduling for real time systems, in: *Proceedings of the SPECTS 2002-2002 International Symposium on Performance Evaluation of Computer and Telecommunication Systems*, 2002.
- [28] J. Goossens, V. Berten, Gang FTP scheduling of periodic and parallel rigid real-time tasks, 2010, *arXiv preprint arXiv:1006.2617*.
- [29] V. Berten, P. Courbin, J. Goossens, Gang fixed priority scheduling of periodic moldable real-time tasks, in: *5th Junior Researcher Workshop on Real-Time Computing*, 2011, pp. 9–12.
- [30] Z. Dong, C. Liu, Analysis techniques for supporting hard real-time sporadic gang task systems, *Real-Time Syst.* 55 (3) (2019) 641–666.
- [31] M. Bertogna, M. Cirinei, Response-time analysis for globally scheduled symmetric multiprocessor platforms, in: *Proceedings of IEEE Real-Time Systems Symposium*, 2007, pp. 149–160.
- [32] N. Audsley, A. Burns, M. Richardson, A. Wellings, Hard real-time scheduling: the deadline-monotonic approach, in: *Proceedings of the IEEE Workshop on Real-Time Operating Systems and Software*, 1991, pp. 133–137.
- [33] C. Liu, J. Layland, Scheduling algorithms for multi-programming in a hard-real-time environment, *J. ACM* 20 (1) (1973) 46–61.
- [34] E. Bini, G. Buttazzo, Measuring the performance of schedulability tests, *Real-Time Syst.* 30 (1–2) (2005) 129–154.